T E C H N I C A L D I A G N O S T I C R E P O R T

Rechargement du moteur de prédictions VOLTRIX

Diagnostic technique et options de résolution

Application : VOLTRIX — Next.js, calculs exécutés dans le navigateur

Périmètre : écran d'accueil, navigation vers une fiche match ou combat

Date d'analyse : 11 novembre 2026

Statut : diagnostic — aucune modification de code effectuée

Table des matières

Synthèse exécutive 1

1. Contexte et symptôme constaté 1

    1.1 Scénario observé 2

    1.2 Impact et périmètre 2

2. Méthode de diagnostic 2

3. Diagnostic : cause racine et mécanisme 2

    3.1 Cause racine : l'écran d'accueil est démonté à chaque navigation 3

    3.2 La chaîne de rechargement déclenchée au retour 3

    3.3 Facteurs amplificateurs 3

    3.4 Ce qui fonctionne déjà correctement 3

4. Options de résolution 4

    4.1 Option A — Conserver l'accueil monté sous la fiche (priorité 1) 4

    4.2 Option B — Cache mémoire de la journée 4

    4.3 Option C — État global de l'application 4

    4.4 Option D — Persistance session des listes 5

    4.5 Option E — Calculs dans un fil séparé (Web Worker) 5

5. Recommandation et plan d'action 5

    5.1 Recommandation 6

Conclusion 6

Remarque : cette table des matières est générée par champs. Après toute modification du document, cliquer avec le bouton droit sur la table puis choisir « Mettre à jour les champs » pour actualiser les numéros de page.

Synthèse exécutive

Ce rapport diagnostique l'anomalie suivante : après consultation d'une fiche match ou combat, le retour à l'écran d'accueil déclenche un rechargement complet des données, d'une durée minimale de trente secondes, alors même que le moteur de prédictions venait de terminer ses calculs.

L'analyse du code source établit que la cause est **architecturale et non statistique**. L'application ne maintient à l'écran que le sommet de sa pile de navigation : ouvrir une fiche démonte intégralement l'écran d'accueil et détruit son état (listes de matchs, pronostics affichés, filtres, date, position de défilement). Au retour, l'écran se reconstruit à vide et relance l'ensemble de la chaîne de chargement. Un facteur amplificateur aggrave le délai dès que l'absence dépasse une minute : la durée de vie du cache des scores du jour est fixée à soixante secondes, ce qui transforme le rechargement en un véritable re-scan réseau de plusieurs dizaines de requêtes. Les prédictions elles-mêmes sont correctement persistées en base locale pendant quinze minutes : le délai perçu provient donc du rechargement des données, et non du recalcul des pronostics.

La recommandation prioritaire est de **conserver l'écran d'accueil monté sous la fiche détaillée** (empilement des écrans), complétée par un cache mémoire de la journée. Le retour attendu passe alors à moins d'une seconde, sans aucune requête réseau lors des retours rapides. Conformément à la demande, aucune modification de code n'a été effectuée à ce stade.

1. Contexte et symptôme constaté

1.1 Scénario observé

Le flux concerné est le suivant : l'utilisateur ouvre l'application ; le moteur de prédictions termine ses calculs et affiche les cartes de matchs et de combats ; l'utilisateur ouvre une fiche (match de football ou combat de MMA) et y reste environ vingt secondes ; en revenant à l'accueil, il constate un rechargement complet — squelettes de chargement, puis reconstruction progressive des listes — d'une durée d'au moins trente secondes.

1.2 Impact et périmètre

Cet effet dégrade directement l'expérience : navigation perçue comme lente, perte de la position de défilement et du filtre actif à chaque consultation d'une fiche, consommation réseau inutile sur mobile, et risque d'abandon de la consultation multi-fiches. Le périmètre couvert par ce diagnostic est l'écran d'accueil et la navigation vers les écrans de détail.

Précision importante : la durée de vingt secondes passée sur la fiche n'est pas la cause du problème. La perte d'état se produit à l'instant de la navigation, quelle que soit la durée de la consultation. En revanche, au-delà d'environ une minute d'absence, l'expiration du cache des scores du jour (durée de vie : soixante secondes) rend un re-scan réseau complet inévitable et allonge d'autant le rechargement.

2. Méthode de diagnostic

Le diagnostic repose sur une analyse statique du code source de l'application, menée par lecture directe des fichiers concernés, sans exécution du moteur ni modification du code, conformément à la demande. Les fichiers examinés sont : la racine de navigation (VoltrixApp.tsx), l'écran d'accueil (HomeScreen.tsx), la couche de chargement du football (lib/foot/bridge.ts et lib/foot/espn.ts), la chaîne d'enrichissement du MMA (lib/mma-bridge.ts) et la base locale de pronostics (lib/pred-db.ts). Les numéros de ligne cités correspondent à l'état du code à la date d'analyse. Chaque constat est rattaché à un emplacement précis du code ; aucune donnée n'a été extrapolée.

3. Diagnostic : cause racine et mécanisme

3.1 Cause racine : l'écran d'accueil est démonté à chaque navigation

La racine de navigation gère une pile d'écrans, mais ne rend à l'écran que l'élément situé au sommet de cette pile (VoltrixApp.tsx, lignes 127 à 160). Dès l'ouverture d'une fiche match ou combat, l'écran d'accueil est **démonté** : la totalité de son état local (HomeScreen.tsx, lignes 371 à 397) est détruite — listes de matchs et de combats, pronostics affichés, filtre actif, date sélectionnée, position de défilement. Au retour, le composant se remonte à vide et son effet de chargement (lignes 485 à 487) s'exécute de nouveau, systématiquement.

3.2 La chaîne de rechargement déclenchée au retour

La fonction de chargement commence par remettre explicitement les données à zéro (données de matchs à null, pronostics à vide — HomeScreen.tsx, lignes 454 à 459), ce qui affiche les squelettes. Elle relance ensuite le scan complet des compétitions de football en trois vagues de priorité, avec quatre requêtes parallèles (lib/foot/bridge.ts, lignes 84 à 129), puis rejoue l'intégralité de la chaîne MMA : événements, photos et drapeaux des combattants, formes récentes et pronostics express (lib/mma-bridge.ts, lignes 357 à 466). Lorsque les caches de bas niveau sont encore valides, seule l'orchestration est rejouée ; lorsqu'ils ont expiré, chaque étape repasse par le réseau.

3.3 Facteurs amplificateurs

Quatre facteurs allongent le rechargement. **Premièrement**, la durée de vie du cache des scores du jour est de soixante secondes (lib/foot/espn.ts, ligne 25) : passé ce délai, le re-scan devient un véritable re-scan réseau de plusieurs dizaines de requêtes, ce qui explique l'ordre de grandeur des trente secondes observées. **Deuxièmement**, la donnée agrégée du jour (listes finales prêtes à afficher) n'est mise en cache nulle part : seules les réponses réseau brutes disposent de durées de vie. **Troisièmement**, en environnement de développement, le mode StrictMode de React exécute chaque effet deux fois, ce qui double la durée perçue. **Quatrièmement**, les vagues de scan s'exécutent séquentiellement entre priorités, ce qui borne la vitesse de reconstruction.

3.4 Ce qui fonctionne déjà correctement

Un point mérite d'être souligné : les pronostics de football et de MMA sont déjà persistés dans une base locale IndexedDB, avec une durée de validité de quinze minutes pour les événements à venir, de dix minutes en direct et de six heures une fois terminés (HomeScreen.tsx, lignes 45 à 53 et 519 à 564). Tant que ces entrées sont valides, **le moteur statistique ne recalcule rien au retour** : les pronostics sont relus instantanément depuis la base locale. Le délai de trente secondes perçu provient donc du re-chargement et de la ré-hydratation des listes de matchs et de combats, et non du recalcul des prédictions. Le problème est réel côté données, mais ne remet pas en cause le travail déjà accompli côté persistance des calculs.

4. Options de résolution

Cinq options ont été identifiées. Elles sont décrites ici de manière conceptuelle, sans implémentation, conformément à la demande.

4.1 Option A — Conserver l'accueil monté sous la fiche (priorité 1)

Modifier uniquement la racine de navigation : au lieu de remplacer l'écran courant, empiler la fiche détaillée par-dessus un écran d'accueil maintenu en vie — le modèle des applications mobiles natives, où la vue précédente reste montée sous la nouvelle. L'accueil n'est jamais démonté : le retour devient instantané, l'état, le filtre et la position de défilement sont intacts, et aucune requête n'est relancée. Effort : faible (un seul fichier concerné). Risque : faible (empreinte mémoire marginale).

4.2 Option B — Cache mémoire de la journée

Conserver le résultat agrégé du jour (listes de football et de MMA, date) dans une structure vivant hors du cycle de vie du composant, de sorte qu'elle survive aux démontages pendant toute la session. Au remontage, l'écran s'affiche immédiatement depuis ce cache, puis se rafraîchit en arrière-plan si nécessaire. Effort : faible. Cette option complète l'option A et couvre les cas où l'accueil serait malgré tout démonté.

4.3 Option C — État global de l'application

Déplacer les données partagées (listes, pronostics, filtres, date) vers un magasin d'état monté à la racine de l'application, indépendant de la navigation. C'est la solution la plus pérenne si l'application gagne de nouveaux écrans partageant les mêmes données, mais elle suppose une refonte de l'organisation de l'état. Effort : moyen.

4.4 Option D — Persistance session des listes

Étendre le mécanisme de base locale déjà en place pour les pronostics aux listes du jour, avec une validité courte (deux à cinq minutes). Cette option couvre le cas supplémentaire du rechargement complet du navigateur, qu'aucune des options précédentes ne traite. Effort : faible.

4.5 Option E — Calculs dans un fil séparé (Web Worker)

Déplacer les calculs lourds du moteur vers un fil d'exécution séparé. Cette option ne supprime pas le rechargement, mais élimine tout gel de l'interface pendant les phases de calcul. Elle constitue une amélioration de confort indépendante. Effort : moyen à élevé. À noter : une déportation des calculs vers un serveur a été écartée d'emblée, car contraire au principe d'une application entièrement exécutée dans le navigateur.

Le tableau suivant compare les cinq options.

Tableau 1 : Comparatif des cinq options de résolution

Option	Effort	Impact sur le symptôme	Risque	Délai estimé
A — Accueil conservé monté	Faible	Total : retour instantané, état conservé	Faible	1 à 2 heures
B — Cache mémoire de journée	Faible	Fort : affichage immédiat au remontage	Faible	Environ 1 heure
C — État global (store)	Moyen	Fort, mais refonte de l'architecture d'état	Moyen	3 à 5 heures
D — Persistance session	Faible	Moyen : couvre le rechargement du navigateur	Faible	Environ 1 heure

Option	Effort	Impact sur le symptôme	Risque	Délai estimé
E — Web Worker	Moyen à élevé	Indirect : supprime le gel, pas le rechargement	Moyen	3 à 5 heures

Les options A et B sont cumulatives : la première supprime le déclenchement du problème, la seconde en garantit la conséquence bénigne si le démontage survient malgré tout. Les options C, D et E répondent à des cas d'usage supplémentaires (croissance de l'application, rechargement navigateur, fluidité des calculs).

5. Recommandation et plan d'action

5.1 Recommandation

La priorité est donnée à **l'option A** : elle supprime la totalité du symptôme dans le flux décrit, pour un effort limité à un fichier. Elle est complétée par l'option B en filet de sécurité et par l'option D pour couvrir le rechargement du navigateur. L'option C est recommandée à moyen terme si l'application s'enrichit de nouveaux écrans ; l'option E reste une amélioration de confort à planifier séparément.

Le plan d'action ci-dessous récapitule les interventions, leur priorité, leur effort estimé, leur propriétaire et le résultat attendu.

Tableau 2 : Plan d'action proposé

Action	Priorité	Effort	Propriétaire	Résultat attendu
A — Empilement des écrans (accueil maintenu)	P1	1 à 2 heures	Développeur VOLTRIX	Retour inférieur à 1 seconde ; zéro requête réseau au retour rapide ; défilement conservé
B — Cache mémoire de la journée	P2	Environ 1 heure	Développeur VOLTRIX	Affichage immédiat après tout démontage de l'accueil
D — Persistance session des listes	P3	Environ 1 heure	Développeur VOLTRIX	Rechargement du navigateur instantané pour des données de moins de 5 minutes
E — Calculs en Web Worker	P4	3 à 5 heures	Développeur VOLTRIX	Interface fluide pendant les calculs du moteur

Trois critères mesurables valideront la correction : **durée du retour depuis une fiche inférieure à une seconde ; aucune requête réseau lors d'un retour après une absence de**

moins d'une minute ; conservation intégrale de la position de défilement et du filtre actif.

Conclusion

Le problème rapporté est un défaut architectural précisément localisé : la navigation par remplacement d'écran détruit l'état de l'accueil à chaque consultation d'une fiche, et la chaîne de rechargement qui s'ensuit est allongée par une durée de cache courte sur les scores du jour. Il n'est imputable ni au temps passé sur la fiche, ni au moteur statistique, dont les résultats sont déjà persistés quinze minutes. Les correctifs identifiés sont simples, progressifs et cumulables, l'option A suffisant à elle seule à supprimer le symptôme dans le flux décrit. Conformément à la demande, aucune modification du code n'a été réalisée ; la prochaine étape est la validation des options retenues, puis la mise en œuvre de l'option A.